

Problema especial Programación 2023

Algoritmos de aprendizaje automático supervisados:

A lo largo de este curso hemos aprendido a diseñar una gran cantidad de programas, que cumplen distintos objetivos, pero todos han tenido algo en común: le hemos dicho a nuestro programa exactamente qué es lo que tiene que hacer y en qué orden debe hacerlo. En este sentido el procesador es “tonto”, solo puede ejecutar las operaciones que le programamos en exactamente el orden que lo hacemos. Muchas veces las cosas no andaban, pero la respuesta siempre estuvo en el mismo lugar: el procesador está haciendo exactamente lo que le dijimos que tenía que hacer, solo que este conjunto de instrucciones era erróneo.

Esta característica de las computadoras está muy bien, pero tiene un problema: hay cosas que no sabemos exactamente como se hacen, simplemente las hacemos. Por ejemplo, ¿sabrías como decirle a una computadora como reconocer un rostro?, ¿o como diferenciar si una pregunta es irónica o no?, ¿o como diseñar un automóvil autónomo? Para diseñar programas que puedan realizar este tipo de tareas es que ha surgido lo que se conoce como aprendizaje automático, aprendizaje de computadora o “machine learning”.

Existen muchos tipos de algoritmos de aprendizaje automático. En este trabajo nos vamos a concentrar en los algoritmos de aprendizaje supervisado. Estos constan de dos etapas: la fase de entrenamiento y la fase de clasificación. Durante la fase de entrenamiento se provee al sistema de una serie de entradas, llamados datos de entrenamiento, o “training data”, cuyas salidas se conocen. El algoritmo estudia los atributos de este set de entrenamiento y genera con ellos un vector de características (o “feature vector”). Una vez finalizada esta etapa el algoritmo sabe que una determinada combinación de características (un determinado vector) da un determinado resultado. Luego sigue la etapa de clasificación. En esta se le provee al algoritmo de nuevas entradas cuyas salidas se desconocen. La idea es que usando lo que “aprendió” en la etapa de entrenamiento pueda determinar cuál es la salida más probable de este nuevo vector de características.

Por ejemplo, supongamos que queremos diseñar un algoritmo que diferencie la cara de dos personas, digamos Juan y Pedro. Nuestro set de entrenamiento serían N fotos de las caras de Juan y de Pedro, etiquetadas con sus nombres. Con estas caras el algoritmo debe armar su vector de características. Por simplicidad supongamos que el algoritmo solo mira el tamaño de la nariz (T) y el color de los ojos (O) de las fotos, es decir un vector bidimensional (T, O). Con cada foto se arma un vector bidimensional, es decir que nuestro set de entrenamiento son N vectores bidimensionales $\{T_i, O_i\} i=0 \dots N-1$. Sabemos que cada vector da un determinado resultado Y_i que sería en este caso Juan o Pedro. Entonces el algoritmo busca *de alguna manera* una regla que lleve el conjunto de vectores $\{T_i, O_i\}$ a Y_i para todas las muestras disponibles. En nuestro ejemplo eso podría ser tan fácil como decir “si el tamaño de la nariz es mayor a 3cm y el color de los ojos es azul entonces es Pedro, sino es Juan”.

Una vez obtenida la regla que lleva nuestro conjunto de entradas a las salidas deseadas pasamos a la etapa de clasificación. En esta le damos una nueva muestra a nuestro algoritmo (en este caso una nueva foto) y usando la regla que encontró en la etapa de entrenamiento este va a generar la salida que más probablemente sea la correcta.

Hasta ahora nos hemos saltado convenientemente la parte de cómo hace el algoritmo para generar la regla que lleva el conjunto de puntos del set de entrenamiento a las salidas deseadas. Para esto existen muchos algoritmos distintos, en esta ocasión nos vamos a concentrar en uno de los más básicos llamado K vecinos más cercanos (KNN por su acrónimo en inglés).

K vecinos más cercanos:

La idea detrás del algoritmo KNN es muy sencilla. Cada elemento del set de entrenamiento se piensa como un punto de un espacio M-dimensional, donde M es la cantidad de atributos que miramos para armar nuestro vector de características (en el ejemplo de los rostros hablamos de un espacio bidimensional). Luego se define una métrica para calcular la distancia entre dos puntos de este espacio. Puede ser por ejemplo la distancia euclidiana entre dos puntos en un espacio M dimensional:

$$d(p, q) = \sqrt{\sum_{i=1}^M (p_i - q_i)^2}$$

Al llegar una nueva muestra obtenemos su vector de características y calculamos a que distancia se encuentra de todas las muestras disponibles en el set de entrenamiento. Tomamos las K muestras más cercanas (donde K es un parámetro que fijamos nosotros) y nos fijamos a que salidas corresponden. Luego el proceso es “democrático” si de estas K muestras hay más que corresponden a la salida Y_1 listo, entonces la nueva muestra se clasifica como Y_1 .

Consigna:

La consigna de este trabajo es diseñar un algoritmo KNN que permita clasificar frases de canciones según sus autores. Para ello el algoritmo debe analizar un conjunto de frases, etiquetadas con sus autores y construir con cada una de ellas un vector de características. Luego al ingresar una nueva frase se comparará el vector de características de esta nueva frase con todas las disponibles, usando la distancia euclídea como métrica, se ordenarán de la más similar (la de menor distancia) a la menos similar y se decidirá el autor de la nueva frase como el que tiene más ocurrencias entre las K frases más cercanas.

Queda contestar una pregunta: ¿Cómo construimos un vector de características a partir de una frase? Existen muchas maneras. En esta ocasión la propuesta es tomar un diccionario con “todas” las palabras del español, digamos M palabras, en orden alfabético. Luego comparamos las M palabras de nuestro diccionario con las de la frase. Para cada palabra evaluamos si la palabra está en la frase, sumando un 1 en ese lugar si está, o 0 si no. El resultado es un vector M dimensional (con muchos ceros).

Vamos a un ejemplo. Imaginemos que nuestro diccionario tiene 10 palabras:

➔ **Diccionario:** {árbol, alto, casa, el, en, fuente, la, patio, tiene, una}

La frase “el árbol alto en el patio” tendrá un vector {1,1,0,2,1,0,0,1,0,0} (son las ocurrencias de las palabras en el orden en que aparecen en el diccionario).

La frase “la casa tiene una fuente” tendrá un vector {0,0,1,0,0,1,1,0,1,1}.

Este enfoque es sencillo, pero tiene varios puntos flojos. En primer lugar, las frases con más palabras tendrán un vector de características más “extenso” (más lejano al origen). Esto se soluciona fácil normalizando cada vector (dividiéndolo por la norma, que es la distancia de un vector cualquiera con el origen). Además, el procedimiento nunca tiene en cuenta el orden de las palabras en la frase, y es imposible tener un diccionario con todas las palabras posibles (por ejemplo, las conjugaciones de verbos no aparecerán). Estos problemas decidiremos ignorarlos por simplicidad de la consigna.

Procedimiento

- 1) **Inicializar el diccionario:** El diccionario debe ser un arreglo de cadenas de caracteres (cada cadena es una palabra del diccionario). Se inicializará desde el archivo "Diccionario.txt", donde cada línea tiene una palabra. Para recorrer este archivo puede utilizarse la función `fscanf`, con el modificador "%s", que lee una palabra cada vez, o el comando `fgets`, recordando que este agrega un salto de línea al final de cada palabra. Si en el diccionario hay `N_diccionario` palabras y cada palabra puede tener `N_palabra` letras el diccionario será un arreglo:

➤ **char diccionario [N_diccionario][N_palabra];**

```
i=0
mientras (no se llegue al fin del archivo)
    diccionario[i] = leer palabra()
    i++
fin_mientras
N_diccionario =i
```

- 2) **Inicializar el set de entrenamiento:** El set de entrenamiento son frases de cada autor que queremos diferenciar, y se inicializará a partir de los archivos "la_renga_training.txt" y "Charly_training.txt". Este set puede ser un arreglo de cadenas de caracteres, con las palabras de cada frase, y un arreglo extra, paralelo al primero, que contendrá el "resultado" de cada conjunto de palabras (el autor de cada frase). Esta última puede ser un arreglo de enteros (0 para un autor, 1 para otro, 2 si hay más, etc.). Si tenemos `N_entrenamiento` frases y cada una puede tener `N_oracion` letras estos vectores serán.

➤ **char training_set [N_entrenamiento][N_oracion];**

➤ **int vector_tipos [N_entrenamiento];**

```
f=abrir("autor1.txt")
i=0
mientras (no se llegue al fin del archivo)
    training_set[i] =leer línea()
    vector_tipos[i]=0
    i++
fin_mientras
cerrar(f)
f=abrir("autor2.txt")
mientras (no se llegue al fin del archivo)
    training_set[i] =leer línea()
    vector_tipos[i]=1
    i++
fin_mientras
cerrar(f)
N_entrenamiento =i
```

- 3) **Obtener el vector de características:** El siguiente paso es comparar cada palabra de cada frase del set de entrenamiento con las disponibles en el diccionario, sumando 1 a la

posición correspondiente cuando estas coincidan. El resultado serán `N_entrenamiento` vectores numéricos, uno por cada frase. Para comparar cadenas de caracteres la librería `string.h` tiene la función `strstr()`, pero tiene un problema, como vimos en la Práctica 2. Cuando una palabra está incluida en otra la función da resultado positivo (ejemplo “boca” va a dar positivo cuando se compare con “bocanada”). Proponemos utilizar la función desarrollada en el ejercicio 20 de la Práctica 2 disponible más abajo. Esta devuelve un número entero que es la cantidad de ocurrencias de la palabra en la frase. Si tenemos `N_entrenamiento` frases y `N_diccionario` palabras estamos hablando de un vector:

➤ `float vector_caracteristicas[N_entrenamiento][N_diccionario];`

```
para j=0 hasta N_entrenamiento
  para i=0 hasta N_diccionario
    vector_caracteristicas[j][i] = strstr_custom( training_set[j] ,diccionario[i]);
  fin para
fin para
```

- 4) **Normalizar vectores de características:** Cada vector de características debe normalizarse, dividiendo cada componente por su norma, entendida como la distancia euclídea de el vector al origen.

```
para j=0 hasta N_entrenamiento
  norma = 0
  para i=0 hasta N_diccionario
    norma += pow ( vector_caracteristicas[j][i] , 2)
  fin para
  norma = sqrt(norma)
  para i=0 hasta ndic
    vector_caracteristicas[j][i] /= norma
  fin para
fin para
```

- 5) **Procesar nueva frase:** A partir de este punto empieza la fase de clasificación. Ante una nueva frase se debe seguir el mismo procedimiento descrito: separarla en palabras, y generar el vector de características normalizado utilizando el diccionario. La única diferencia es que ahora no sabemos el resultado, sino que es lo que queremos averiguar.
- 6) **Calcular la distancia de la nueva frase a los elementos del set de entrenamiento:** En este punto se calcula la distancia del vector de características de la nueva frase a todas las del set de entrenamiento, utilizando la distancia euclídea como métrica. Se puede utilizar la implementación del cálculo de distancia provista más abajo.
- 7) **Ordenar el set de entrenamiento según las distancias:** Ahora que tenemos todas las distancias calculadas solo queda ordenar los vectores de características disponibles y ver a que salidas corresponden las K más cercanas. Luego se decidirá que la frase pertenece a la salida que más ocurrencias tenga entre esas K. Para ordenar el set de entrenamiento se puede utilizar el algoritmo de la burbuja, visto en la Práctica 3. En este paso hay que recordar que el set de entrenamiento, el vector de resultados y los vectores de características están alineados, hay que ordenarlos todos según los valores calculados de distancias.

Notas: Como estamos trabajando con vectores de dimensiones elevadas los vectores deben declararse como globales; si se declaran dentro del “main” el programa no reserva la cantidad de memoria necesaria para las operaciones.

A veces trabajar con el diccionario de todas las palabras puede ser engorroso. Para verificar que los algoritmos funcionen correctamente se provee un diccionario reducido (“Diccionario_corto.txt”). Se pueden fabricar frases con las palabras de este diccionario y verificar que todo ande correctamente antes de seguir con la consigna propuesta.

Evaluación:

El trabajo es individual, pero puede ser realizado en grupo. En el caso de ser realizado en grupo, además de especificar con quién se hizo, cada trabajo deberá tener alguna característica particular para diferenciar los trabajos individuales. Deberá entregarse la carpeta del proyecto de Codeblocks, o el IDE utilizado, que incluya un archivo README.txt donde se especifique:

- Nombre y número de alumno
- Email
- Fecha de la última modificación del código
- Breve descripción sobre cómo usar el programa
- Breve descripción de la organización del código

Y si aplica:

- Grupo de trabajo (con nombre y número de alumno de cada integrante)
- Aclaración de la característica distintiva de su entrega para distinguirse del trabajo en grupo.

El trabajo se defenderá individualmente en forma oral. La fecha límite para su entrega es la fecha del flotante (4 de agosto de 2023). El programa presentado debe compilar y debe poder predecir correctamente, utilizando el método propuesto, los autores de las frases disponibles en “la_renga_classify.txt” y “Charly_classify.txt”, para $K=5$.

El programa debe estar correctamente modularizado y escrito en c (programación estructurada) o c++ (programación orientada a objetos) según su preferencia.

Ayudas:

- **Función strstr_custom:** Implementación de str_str que no falla cuando una palabra es parte de otra.

```
int strstr_custom(char cadena[],char palabra[])
{
    int i=0;
    int j,flag;
    int ocurrencias=0;
    int N = strlen(palabra);

    while(cadena[i] != '\0' )
    {
        if( (i == 0) || (cadena[i] == ' ') )
        {
            if(cadena[i] == ' '){i++;}

            if( (cadena[i] == palabra[0]) || (cadena[i] + 'a'-'A' == palabra[0]) || (cadena[i]) == palabra[0] + 'a'-'A' )
            {
                flag = 1;
                for (j =1;j<N;j++)
                {
                    if(cadena[i+j] != palabra[j] )
                    {
                        flag = 0;
                        break;
                    }
                }

                if( (flag == 1) && ( (cadena[i+N] == ' ') ||(cadena[i+N]) == '\0' ) )
                {
                    ocurrencias++;
                    flag=0;
                }
            }
            i++;
        }
    }
    return ocurrencias;
}
```

- **Cálculo de distancia euclídea:**

```
float calcular_distancia(float vector1[],float vector2[],int dimension)
{
    int i;
    float distancia=0;
    for (i=0;i<dimension;i++)
    {
        distancia = distancia + pow((vector1[i] - vector2[i]),2 );
    }
    distancia = sqrt(distancia);
    return distancia;
}
```

- **Algoritmo de la burbuja:**

```
void bubbleSort(int vector[], int n) {
    int i, j, temp;
    for (i = 0; i < n-1; i++) {
        for (j = 0; j < n-i-1; j++) {
            if (vector[j] > vector[j+1]) {
                // Intercambiar los elementos
                temp = vector[j];
                vector[j] = vector[j+1];
                vector[j+1] = temp;
            }
        }
    }
}
```